

1. Can we run different versions of browsers in Playwright?

Yes, Playwright allows running different versions of browsers, but with some limitations:

- **Default:** Playwright comes bundled with specific versions of Chromium, Firefox, and WebKit.
- **Custom versions:** You can specify a custom executable path using the `executablePath` option to run a different version of a browser (e.g., a specific version of Chrome or Edge).

Example: Run a custom version of Chrome

```
js
CopyEdit
const { chromium } = require('playwright');

(async () => {
  const browser = await chromium.launch({
    executablePath: '/path/to/your/chrome' // Path to custom Chrome
  });
  const context = await browser.newContext();
  const page = await context.newPage();
  await page.goto('https://example.com');
  await browser.close();
})();
```

2. How can we run Edge in Playwright?

Microsoft Edge is Chromium-based, so you can run it using the `chromium` engine with the `executablePath` set to Edge's binary.

Windows Example:

```
js
CopyEdit
const { chromium } = require('playwright');

(async () => {
  const browser = await chromium.launch({
    executablePath: 'C:\\Program Files
(x86)\\Microsoft\\Edge\\Application\\msedge.exe'
  });
  const context = await browser.newContext();
  const page = await context.newPage();
  await page.goto('https://example.com');
  await browser.close();
})();
```

3. Explain Autowaits in Playwright

Auto-waiting is one of Playwright's core features. It automatically waits for elements to be ready before performing actions (like click, fill, type).

How it works:

- Waits for the element to be **attached**, **visible**, and **stable** before acting.
- Applies to methods like `click()`, `fill()`, `type()`, `hover()`, `check()`, etc.
- No need to manually write `waitForSelector()` in most cases.

Example:

```
js
CopyEdit
await page.click('text=Submit'); // Waits for the button to be visible and
enabled
```

4. How to handle Alerts, Windows, and Dialogs in Playwright

Alerts/Dialogs (e.g., alert, confirm, prompt):

Use `page.on('dialog')` to handle browser dialogs.

```
js
CopyEdit
page.on('dialog', async dialog => {
  console.log(dialog.message());
  await dialog.accept(); // or dialog.dismiss()
});

await page.click('#trigger-alert'); // This will open the alert
```

New Windows (Pop-ups):

Use `context.on('page')` or `page.waitForEvent('popup')`.

```
js
CopyEdit
const [popup] = await Promise.all([
  page.waitForEvent('popup'),
  page.click('#open-new-window')
]);
await popup.goto('https://example.com');
```

5. Playwright Fixtures

Fixtures in Playwright (when using Playwright Test) are reusable setups like browser, context, page, or your custom logic, that are automatically injected into tests.

Why use fixtures?

- Reduce code duplication.
- Improve test modularity.
- Easily manage setup/teardown.

Example: Custom fixture with Playwright Test

```
js
CopyEdit
// tests/example.spec.ts
import { test, expect } from '@playwright/test';

test('example using page fixture', async ({ page }) => {
  await page.goto('https://example.com');
  await expect(page).toHaveTitle(/Example Domain/);
});
```

Define custom fixtures:

```
js
CopyEdit
// fixtures/my-fixtures.ts
import { test as base } from '@playwright/test';

const test = base.extend({
  customData: async ({}, use) => {
    const data = { user: 'admin', role: 'QA' };
    await use(data);
  }
});

export { test };
```

Then in your test:

```
js
CopyEdit
import { test } from '../fixtures/my-fixtures';

test('uses custom data', async ({ customData }) => {
  console.log(customData.user); // 'admin'
});
```

Let me know if you want code in a specific language (JavaScript, TypeScript, or Python) or want a PDF/notes version.

You said:

explain wait for

ChatGPT said:

In **Playwright**, `waitFor` is a concept used to **wait for certain conditions or events to happen before proceeding**. While Playwright has **auto-waiting built-in**, sometimes you may still need to use explicit waits.

✓ Common `waitFor` methods in Playwright

1. `page.waitForSelector(selector)`

Waits for an element matching the selector to appear in the DOM.

```
js
CopyEdit
await page.waitForSelector('#login-button'); // waits until the element
appears
```

You can also add options:

```
js
CopyEdit
await page.waitForSelector('#submit', { state: 'visible', timeout: 5000 });
```

States:

- 'attached' – element is in DOM (default)
 - 'visible' – element is visible
 - 'hidden' – element is hidden but in DOM
 - 'detached' – element is removed from DOM
-

2. `page.waitForTimeout(ms)`

Waits for a specific time in milliseconds. Avoid using this unless necessary.

```
js
CopyEdit
await page.waitForTimeout(3000); // wait for 3 seconds
```

⚠️ Not recommended unless absolutely required. Prefer auto-waiting or `waitForSelector`.

3. `page.waitForFunction(fn)`

Waits until the function returns a truthy value in the browser context.

```
js
CopyEdit
await page.waitForFunction(() =>
document.querySelector('#status').innerText === 'Ready');
```

Useful when waiting for some JS-based condition to become true.

4. `page.waitForEvent(event)`

Waits for a specific event on the page (e.g., popup, load, console, etc.)

```
js
CopyEdit
const [popup] = await Promise.all([
  page.waitForEvent('popup'),
  page.click('#open-window')
]);
```

5. ElementHandle.waitForElementState(state)

Waits for a specific state of an element: 'visible', 'hidden', 'enabled', 'disabled', 'stable'.

```
js
CopyEdit
const button = await page.$('#submit');
await button.waitForElementState('visible');
```

⚙️ Summary Table

Method	Purpose
<code>waitForSelector()</code>	Wait for element to appear/hide
<code>waitForTimeout()</code>	Wait for a fixed time (ms)
<code>waitForFunction()</code>	Wait for a JS condition
<code>waitForEvent()</code>	Wait for an event like <code>popup</code> or <code>download</code>
<code>elementHandle.waitForElementState()</code>	Wait for a specific element state

📌 Best Practice

- Use **auto-waits** when possible (e.g., `await page.click('text=Login')`)
 - Use `waitForSelector()` or `waitForFunction()` when you have dynamic content or JS rendering
-

Let me know if you need code examples in Python or TypeScript as well.

You said:

explain how to switch from one page to another page in playwright

ChatGPT said:

In **Playwright**, switching from one page to another usually means one of the following:

✓ 1. Handling and switching to a new window or tab (popup)

When a link or button opens a new page (like `window.open()` or `target="_blank"`), Playwright allows you to **wait for and switch to that new page** using `page.waitForEvent('popup')`.

✓ Example:

```
js
CopyEdit
const [newPage] = await Promise.all([
  page.waitForEvent('popup'), // Waits for new page
  page.click('a[target="_blank"]') // Triggers the new tab
]);

await newPage.waitForLoadState(); // Optional: wait for new page to load
console.log(await newPage.title()); // Now you're working on the new page
```

✓ 2. Switching between multiple pages in the same browser context

If you already have multiple pages open, you can **store references** to them and switch easily.

Example:

```
js
CopyEdit
const context = await browser.newContext();
const page1 = await context.newPage();
await page1.goto('https://example.com');

const page2 = await context.newPage();
await page2.goto('https://google.com');

// Interact with page2
await page2.click('input[name="q"]');

// Switch back to page1
await page1.reload();
```

✓ 3. Navigating from one URL to another in the same page

If you're not switching tabs but simply changing the page URL:

Example:

```
js
CopyEdit
await page.goto('https://example.com');
// ... do something

await page.goto('https://google.com');
// now you're on another page (same tab)
```

✓ 4. From a link that opens a new page via JS (e.g., `window.open`)

Same as first case but triggered via script:

```
js
CopyEdit
const [newPage] = await Promise.all([
  page.waitForEvent('popup'),
  page.evaluate(() => window.open('https://bing.com'))
]);
await newPage.waitForLoadState();
```

□ Summary:

Scenario	What to Use
Clicking link that opens new tab	<code>page.waitForEvent('popup')</code>
Manually opening new tab	<code>context.newPage()</code>
Switching between existing pages	Store page references
Navigating same page to new URL	<code>page.goto('url')</code>

Let me know if you'd like the same explanation in **Python** or **TypeScript**, or want a working demo file

To **return to the original page** after verifying the new window's title in Playwright, you can save a reference to the original page before switching to the new one. Here's how you can modify the code:

```
javascript
CopyEdit
Then('{string}: Switch and Verify the page title as {string}', async
function (this: CustomWorld, _step, title) {
  init(this);
  const originalPage = this.page; // Save reference to original page

  await this.page.waitForLoadState('load');
  await expect(this.page).toHaveTitle(title, { timeout: 70_000 });
  await this.page.waitForLoadState('load');

  for (const p of getContext(this).pages()) {
    if ((await p.title()).includes(title)) {
      this.page = p;
      await this.page.bringToFront();

      // Perform verification or any logic here if needed
      break;
    }
  }

  // Return to the original page
  this.page = originalPage;
  await this.page.bringToFront();
});
```

What's Added:

- `const originalPage = this.page;` stores the current page before switching.
- After the new tab/window logic, `this.page = originalPage;` switches back.
- `await this.page.bringToFront();` brings the original page to the front.

Let me know if you want to perform actions in the new tab before returning!

Then('{string}: Switch and Verify the page title as {string}', async function(this: CustomWorld, _step, title) {

```
    init(this);

    await this.page.waitForLoadState('load');

    await expect(this.page).toHaveTitle(title, { timeout: 70_000 });

    await this.page.waitForLoadState('load');

    for (const p of getContext(this).pages()) {
        if ((await p.title()).includes(title)) {
            this.page = p;
            await this.page.bringToFront();
        }
    }
});
```

Then(/(.*)Generic: User clicks '(.*)' text/, async function(this: CustomWorld, _p, text) {

```
    init(this);

    await thaaBasePage.clickElementWithGivenText(text);

});
```